

# Proyecto #1 — Competencia de Modelación (MLP · Ames House Prices)

CC3092 Deep Learning y Sistemas Inteligentes

Ernesto Ascencio 23009

**Repositorio:** <https://github.com/AscencioSIUU/deepLearning/tree/main/projects/proy-1-competencia>

## 1. Análisis exploratorio de datos (EDA)

### 1.1 Dimensiones y tipos de variables

`train.csv` tiene 1168 filas × 81 columnas. La variable objetivo es `SalePrice` (continua, USD). De las 79 features restantes (se descarta `Id`, identificador sin señal): 36 numéricas (áreas, conteos, años) y 43 categóricas de texto. Dentro de las categóricas, 20 son en realidad **ordinales** con un orden natural (calidad: `Po<Fa<TA<Gd<Ex`; exposición de sótano; terminación de garaje; pendiente del terreno; etc.) y 23 son **nominales puras** sin orden (`Neighborhood`, `SaleType`, `Exterior1st`, ...).

### 1.2 Estadísticas descriptivas

`SalePrice`: media \$181,442, mediana \$165,000, desviación estándar \$77,264, rango [\$34,900, \$745,000]. Las features numéricas con mayor correlación con el precio son `OverallQual` (calidad general de construcción), `GrLivArea` (área habitable), `GarageCars`/`GarageArea`, `TotalBsmtSF` y `1stFlrSF` — todas relacionadas con tamaño y calidad de la construcción, consistente con la intuición de mercado inmobiliario. `OverallCond` (condición) correlaciona casi nulo, sugiriendo que la calidad de construcción pesa más que el estado de mantenimiento.

### 1.3 Valores nulos, outliers e inconsistencias

19 columnas tienen nulos. En este dataset, `NA` casi siempre significa “**la casa no tiene esa característica**” (sin piscina, sin garaje, sin sótano) según el diccionario de datos original de Ames — no es un dato faltante real. Tratamiento:

- `PoolQC` (1162 nulos), `MiscFeature` (1122), `Alley` (1094), `Fence` (935), `FireplaceQu` (547), y las columnas `Garage*/Bsmt*/MasVnrType`: nulo → categoría explícita “None”. No se eliminan columnas pese al alto % de nulos, porque el nulo mismo es informativo (ausencia de la característica correlaciona con precio).
- `LotFrontage` (217 nulos, numérica): sin ausencia estructural clara → imputación por mediana (supuesto MCAR).
- `Electrical` (1 nulo): imputación por moda.

**Outliers:** se identificaron 2 casas con `GrLivArea` > 4000 y `SalePrice` < \$300,000 (`Id` 524 y 1299) — ventas atípicas documentadas del dataset original de Ames que rompen la relación esperada área↔precio. Se eliminan del set de entrenamiento.

## 1.4 Visualizaciones

Ver docs/figs/: saleprice\_dist.png (distribución de SalePrice, sesgo positivo marcado, y su versión log1p mucho más simétrica), outliers\_scatter.png (GrLivArea/OverallQual vs. SalePrice, outliers visibles), correlaciones.png (top features numéricas correlacionadas con el precio).

## 1.5 Decisiones de preprocesamiento (resumen)

| Decisión                                                                        | Justificación                                                   |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Eliminar Id                                                                     | identificador, sin señal                                        |
| Eliminar 2 outliers GrLivArea>4000 & SalePrice<\$300k                           | ventas atípicas conocidas del dataset                           |
| Target $\rightarrow \log_{1p}(\text{SalePrice})$                                | corrige sesgo positivo; RMSE en log $\approx$ error relativo    |
| Nulos en categóricas de ausencia de feature $\rightarrow$ "None"                | el nulo es informativo                                          |
| LotFrontage $\rightarrow$ mediana; Electrical $\rightarrow$ moda                | únicos nulos sin ausencia estructural clara                     |
| 20 ordinales de calidad $\rightarrow$ OrdinalEncoder con orden real por columna | preserva el orden, evita explotar dimensionalidad               |
| 23 nominales puras $\rightarrow$ one-hot                                        | sin orden natural                                               |
| 36 numéricas $\rightarrow$ StandardScaler                                       | MLP con gradiente converge mejor con features en escala similar |

## 2. Metodología de desarrollo

**Arquitecturas consideradas:** MLPs totalmente conectados con 2–3 capas ocultas, anchura entre 32 y 256 neuronas por capa, activación ReLU, dropout opcional (0–0.3), BatchNorm1d opcional, salida de una neurona (regresión). Se probaron 8 combinaciones distintas (sección 3).

**División de datos:** split 80/20 train/validation con seed fija (42) para el sweep de iteraciones, sin fuga de información (el ColumnTransformer se ajusta solo con train). La config ganadora se validó además con **5-fold cross-validation** (cross\_validate\_config en src/model.py): en cada fold se reajusta el pipeline de preprocesamiento únicamente con los datos de ese fold de train, evitando fuga entre folds, y el modelo se reentrena desde cero.

**Función de pérdida, optimizador, hiperparámetros:** MSELoss sobre el target en escala  $\log_{1p}(\text{SalePrice})$  (evita que las pocas casas caras dominen el gradiente y hace que el error penalice proporcionalmente en toda la escala de precios). Optimizador Adam, learning rate  $1e-3$  (o  $3e-4$  en una variante), batch size 32.

**Regularización:** early stopping por RMSE de validación (paciencia 20–25 épocas, restaura los pesos del mejor punto visto) en todas las configs; dropout, weight decay (L2) y batch normalization se probaron como variantes adicionales (sección 3).

## 3. Resultados de iteraciones

Sweep de 8 configuraciones, cada una variando **un solo eje** respecto a un baseline común, para poder atribuir el efecto de cada cambio:

| Config             | Arquitectura        | Dropout | Weight decay | BatchNorm | lr     | Épocas | Train RMSE (log) | Val RMSE (log) | Val RMSE (USD)  |
|--------------------|---------------------|---------|--------------|-----------|--------|--------|------------------|----------------|-----------------|
| <b>C1_baseline</b> | 64 $\rightarrow$ 32 | 0.0     | 0            | No        | $1e-3$ | 194    | 0.063            | <b>0.204</b>   | <b>\$41,401</b> |

| Config          | Arquitectura | Dropout | Weight decay | BatchNorm | lr   | Épocas | Train RMSE (log) | Val RMSE (log) | Val RMSE (USD) |
|-----------------|--------------|---------|--------------|-----------|------|--------|------------------|----------------|----------------|
| C2_mas_cap      | 256→64→32    | 0.0     | 0            | No        | 1e-3 | 93     | 0.070            | 0.237          | \$73,019       |
| C5_weight_decay | 128→64       | 0.0     | 1e-4         | No        | 1e-3 | 62     | 0.109            | 0.255          | \$89,568       |
| C7_dropout      | 128→64       | 0.2     | 1e-4         | No        | 1e-3 | 64     | 0.277            | 0.267          | \$47,996       |
| C8_lr_bajo      | 128→64       | 0.2     | 1e-4         | No        | 3e-4 | 64     | 0.267            | 0.300          | \$63,974       |
| C3_mas_ancho    | 256→128      | 0.0     | 0            | No        | 1e-3 | 66     | 0.095            | 0.320          | \$340,645      |
| C4_dropout      | 128→64       | 0.3     | 0            | No        | 1e-3 | 40     | 0.497            | 0.328          | \$69,205       |
| C6_batchnorm    | 128→64       | 0.0     | 0            | Sí        | 1e-3 | 49     | 0.356            | 0.522          | \$175,697      |

Curvas de entrenamiento de las configs más relevantes: docs/figs/sweep\_comparacion.png.

**Evidencia de overfitting:** C3 (más ancho, sin regularización) tiene el mayor gap train/val entre las configs “limpias” ( $0.320 - 0.095 = 0.226$ ) y un RMSE en USD desproporcionado (\$340,645) — 256→128 sin dropout/weight\_decay memoriza patrones espurios de las ~930 filas de entrenamiento que no generalizan.

**Evidencia de inestabilidad:** C6 (batchnorm) fue la peor config en general. Con batch\_size=32 sobre un dataset de este tamaño, las estadísticas de batch que normaliza BatchNorm1d son ruidosas, introduciendo inestabilidad en vez de acelerar la convergencia — consistente con la recomendación general de usar batch-norm en datasets grandes con batches más estables.

**Validación cruzada de la config ganadora:** para confirmar que C1 no ganó por ruido del split 80/20, se corrió 5-fold CV reajustando el pipeline en cada fold. Resultado: **val RMSE log =  $0.227 \pm 0.035$ , val RMSE USD = \$43,278  $\pm$  \$9,175** — consistente con el \$41,401 del split original (dentro de 1 desviación estándar), confirmando que la ventaja del baseline es robusta y no un artefacto de un único split.

## 4. Discusión de resultados

**Comparación entre iteraciones:** el baseline (C1, el modelo más simple del sweep) ganó con margen claro. Con 223 features (tras el one-hot) y ~930 filas de entrenamiento efectivas, la razón parámetros/muestras ya es alta incluso con la arquitectura más chica (64→32  $\approx$  16.5k parámetros). Añadir capacidad (C2, C3) sin compensar con regularización solo empeoró el gap train/val. Sorprendentemente, dropout y weight decay (C4, C5, C7) tampoco mejoraron sobre C1 — indicio de que el early stopping por sí solo ya regularizaba lo suficiente, y que el dropout adicional restó capacidad útil antes de que el modelo terminara de aprender la señal (el train\_rmse de C4 quedó en 0.497, muy por encima de las demás — subentrenado en las pocas épocas que corrió antes del early stop).

**Análisis de errores del modelo final:** sobre el propio set de entrenamiento (desempeño optimista, no generalización), el RMSE es \$13,037 excluyendo los 2 outliers conocidos. Los residuos tienen media -\$5,497 (el modelo tiende a **subestimar** ligeramente el precio en promedio) y desviación estándar \$11,827. Los errores más grandes concentran casas de valor alto o atípico: una casa en OldTown de \$475,000 se sobreestima en \$122,146; casas pequeñas (958–1795 ft<sup>2</sup>) en Somerset/Names se sobreestiman por \$50k–\$70k. El patrón sugiere que el modelo generaliza peor en los extremos de la distribución de precio/tamaño que en el rango típico (\$130k–\$215k), donde está la mayoría de los datos de entrenamiento.

**Fragilidad ante extrapolación:** al validar predict.py sobre train.csv completo (incluyendo los 2 outliers excluidos del entrenamiento), la casa Id=1299 (GrLivArea=5642, precio real \$160,000) recibió una predicción de **\$3.88M** — un error de +\$3.7M en un solo punto. El modelo nunca vio casas con esa área durante entrenamiento; en log-espacio el error es moderado, pero expm1 lo amplifica exponencialmente al volver a USD. Esto es una limitación real y relevante: si el held-out de competencia contiene casas fuera del rango de train.csv, el RMSE en USD puede dispararse por un único punto atípico.

**Trade-off complejidad/generalización:** el hallazgo central del sweep es que, para este dataset (~1000 filas, 223 features tras codificación), **menos es más:** el modelo más simple generalizó mejor que todas las variantes con más capacidad o más regularización explícita. Esto es consistente con la teoría de aprendizaje estadístico — con pocas muestras relativas a la dimensión, la complejidad efectiva del modelo debe mantenerse baja, y el early stopping ya cumple ese rol sin necesitar mecanismos adicionales.

**Limitaciones del enfoque y del dataset:** (1) el sweep completo (8 configs) usó un solo split 80/20, con ruido de muestreo no cuantificado en el ranking de configs cercanas — mitigado solo para la ganadora vía CV posterior, no para las 8 configs completas por presupuesto de tiempo; (2) el one-hot de `Neighborhood` (25 categorías) por sí solo aporta ~25 columnas dispersas, con pocas observaciones por categoría en los barrios menos frecuentes; (3) el modelo extrapola mal fuera del rango de entrenamiento, como se documentó arriba.

## 5. Conclusiones

- **Desempeño final:** RMSE de validación = **\$41,401 USD** (split 80/20 único, 0.2039 en log) y **\$43,278 ± \$9,175 USD** (5-fold CV,  $0.227 \pm 0.035$  en log). Ambas estimaciones son consistentes entre sí, lo que da confianza en que ~\$41k–\$43k es la generalización esperada en el held-out de competencia, dado que el modelo final se reentrenó sobre el 100% de `train.csv` con el número de épocas óptimo encontrado en el split.
- **Aprendizajes técnicos:** (1) entrenar sobre el target en escala log es determinante en datasets de precios con sesgo positivo, tanto para la optimización como para la interpretabilidad del error; (2) con pocas muestras relativas a la dimensión, un modelo simple con early stopping puede superar variantes con más capacidad o regularización explícita — la regularización “correcta” no es automáticamente “más regularización”; (3) invertir una transformación no lineal (`expm1`) amplifica errores de forma no uniforme, hay que revisar el error en la escala en que realmente importa (USD), no solo en la escala de entrenamiento (log); (4) validar la config ganadora con k-fold CV (no solo con el split original) es barato (~15s en CPU) y da una estimación de generalización con incertidumbre cuantificada, en vez de un solo número puntual.
- **Mejoras futuras:** extender el 5-fold CV a las 8 configs completas del sweep (no solo a la ganadora) para un ranking más robusto; clipping de predicciones a un rango razonable como salvaguarda contra extrapolación catastrófica; feature engineering (edad de la casa, área total combinada) para capturar señal no explícita en las columnas crudas; ensamble de varios modelos (promedio de predicciones) para reducir varianza.

## 6. Enlace al repositorio de GitHub

<https://github.com/AscencioSIUU/deepLearning/tree/main/projects/proy-1-competencia>

Contiene el código completo: EDA y desarrollo (`proy1_mlp.ipynb`, generado por `build_nb.py`), preprocesamiento (`src/preprocessing.py`), modelo, training loop y cross-validation (`src/model.py`), entrenamiento final (`train.py`, serializa el pipeline con el módulo estándar `pickle` y los pesos con `torch.save`) y predicción/evaluación (`predict.py`), con instrucciones de reproducción en el `README.md` del proyecto.